



**UNIVERSITY OF FRONTIER TECHNOLOGY,
BANGLADESH**

Department of Cyber Security Engineering

Lab Report

Name of the report : CRUD operation in laravel

Course title : Android and web Application Development sessional

Course code : PROG 212

Submitted by:

Name : Ivana Islam Bubly

ID : 2304034

Level : 02

Term : 01

**Department of Cyber Security
Engineering, UFTB**

Submitted to:

Md. Abdullah Lecturer,

**Department of Cyber Security
Engineering**

**University of Frontier Technology,
Bangladesh**

Table of Contents

Section	Title	Page
1	Objective	4
2	Introduction	4
3	Tools and Technologies Used	4
4	Environment Setup (XAMPP and MySQL)	4
4.1	Starting the MySQL Server via XAMPP	4
4.2	Accessing phpMyAdmin	5
4.3	Creating the MySQL Database	5
5	Laravel Project Configuration	6
5.1	Locating the .env Configuration File	6
5.2	Configuring MySQL Database Connection	6
6	Building the Laravel MVC Structure	7
6.1	Creating the Model, Migration and Controller	7
6.2	Defining the Database Schema (Migration)	7
6.3	Running Migrations on MySQL	8
6.4	Defining Resource Routes	8

Section	Title	Page
6.5	Implementing the Controller	9
6.6	Creating Blade View Files	9
7	Verifying Tables in phpMyAdmin	11
8	Testing CRUD Operations	11
8.1	Read Operation	12
8.2	Create Operation	12
8.3	Verifying Create in phpMyAdmin	13
8.4	Update Operation	13
8.5	Delete Operation	14
9	Conclusion	15

1. Objective

The objective of this lab is to understand and implement the four fundamental CRUD (Create, Read, Update, Delete) operations using the Laravel PHP framework connected to a MySQL database managed via XAMPP and phpMyAdmin. The lab covers the complete development process — from setting up the MySQL environment, creating the Laravel MVC structure (Model, Migration, Controller, Routes, Views), to testing and verifying all CRUD operations both through the browser and directly in the MySQL database using phpMyAdmin.

2. Introduction

CRUD stands for Create, Read, Update, and Delete — the four basic operations used to manage data in any database-driven application. In Laravel, these operations are implemented following the MVC (Model-View-Controller) architecture, where:

- Model — represents the database table and handles data logic using Eloquent ORM.
- View — the Blade template files that display the user interface.
- Controller — handles the application logic and connects the Model with the View.

In this lab, MySQL is used as the database backend, running locally through XAMPP — a popular local development environment that includes Apache and MySQL (MariaDB). phpMyAdmin, a web-based interface bundled with XAMPP, provides a graphical tool to inspect and verify the database records directly. Laravel connects to this MySQL database through the .env configuration file, and Eloquent ORM handles all SQL queries automatically, allowing CRUD operations to be performed using simple PHP method calls without writing raw SQL.

3. Tools and Technologies Used

- Laravel Framework (v13.15.0)
- PHP (v8.5.7)
- XAMPP v3.3.0 — Apache Web Server + MySQL (MariaDB 10.4.32)
- phpMyAdmin v5.2.1 — Web-based database management interface
- Visual Studio Code — Code Editor
- Windows PowerShell — Command Line Interface
- Microsoft Edge — Web Browser for testing

4. Environment Setup (XAMPP and MySQL)

4.1 Starting the MySQL Server via XAMPP

XAMPP Control Panel was launched to start the Apache and MySQL services required for the local development environment. Although the XAMPP panel initially displayed a MySQL startup warning, the MySQL server was successfully started by running the mysqld.exe binary directly from the command line, which confirmed that MariaDB was running correctly on port 3306.

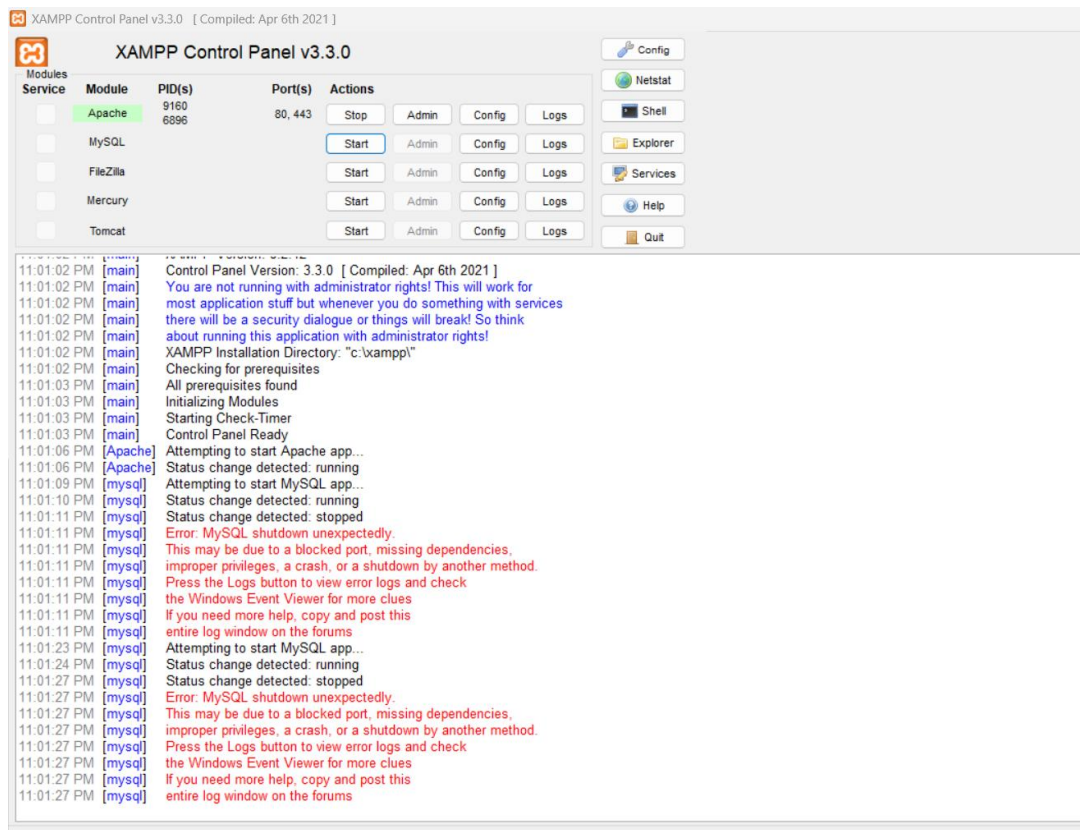


Fig 4.1: XAMPP Control Panel showing Apache running and MySQL startup

4.2 Accessing phpMyAdmin

After the MySQL server started successfully, phpMyAdmin was accessed through the browser at localhost/phpmyadmin. The dashboard confirmed the database server details: MariaDB 10.4.32 running at 127.0.0.1 via TCP/IP, with root as the active user.

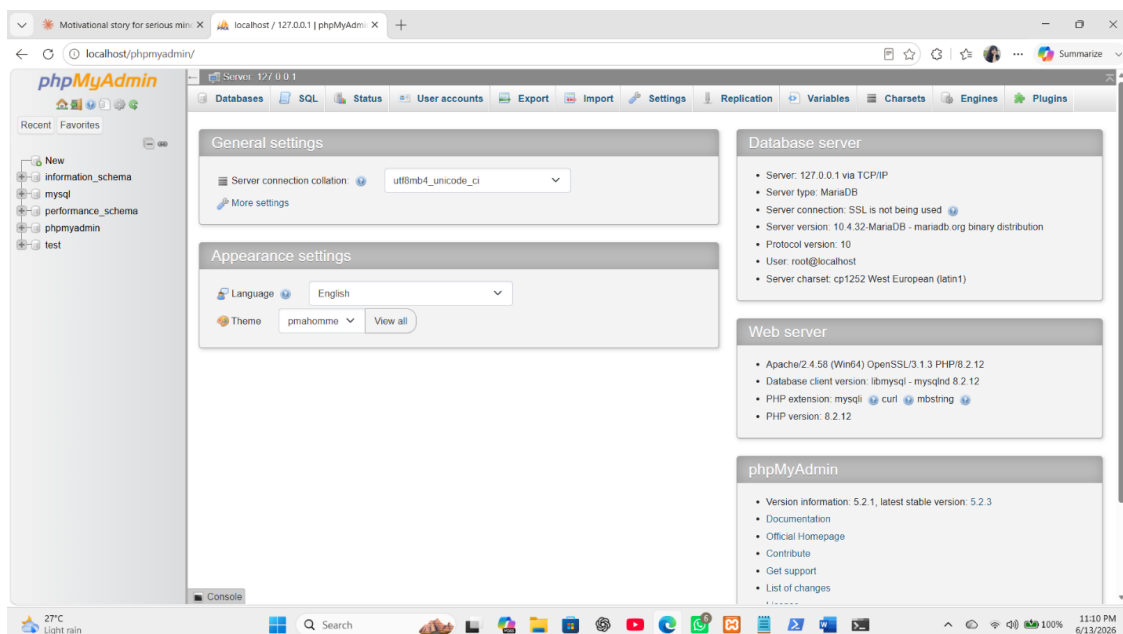


Fig 4.2: phpMyAdmin dashboard confirming MySQL server connection

4.3 Creating the MySQL Database

A new database named 'laravel' was created in phpMyAdmin using the 'Create database' option with the utf8mb4_general_ci collation. This database serves as the primary data store for the Laravel application throughout this lab.

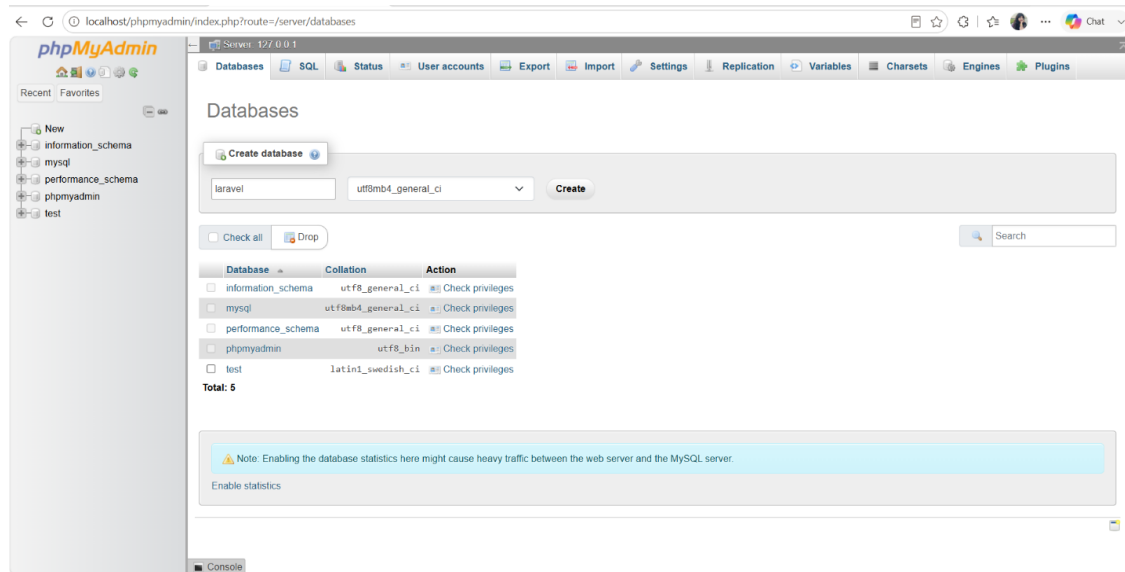


Fig 4.3: Creating the 'laravel' database in phpMyAdmin

5. Laravel Project Configuration

5.1 Locating the .env Configuration File

Laravel uses a .env file in the project root directory to store environment-specific settings including the database connection. The .env file was located in the project folder at C:\Users\DELL\Projects\ivana and opened for editing.

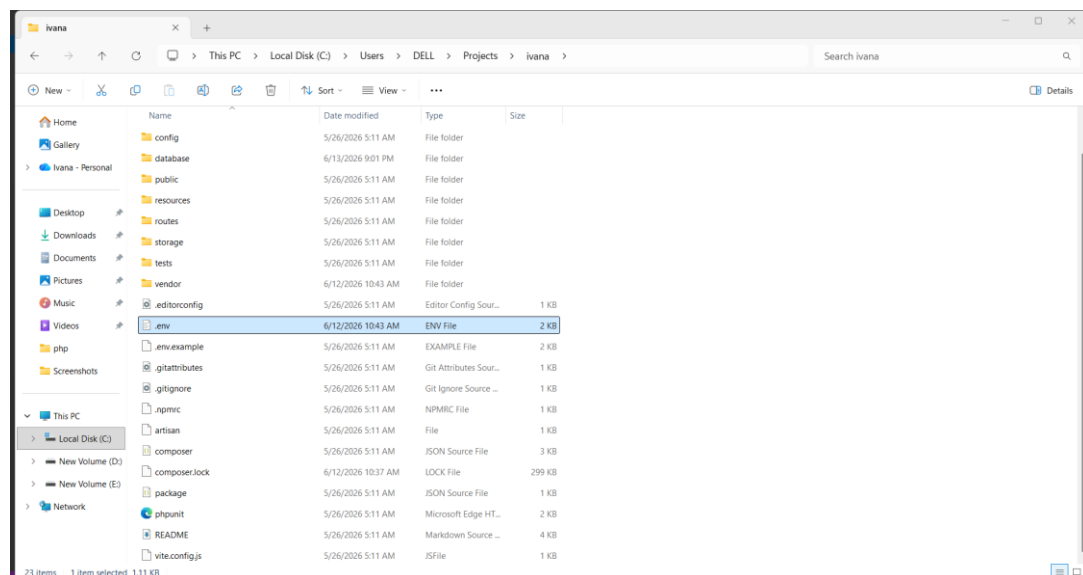


Fig 5.1: Project root directory showing the .env file

5.2 Configuring MySQL Database Connection

The .env file was updated to switch the database connection from the default SQLite to MySQL. The DB_CONNECTION was set to mysql and the host, port, database name, username and password values were configured to match the XAMPP MySQL server settings.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=root
DB_PASSWORD=
```

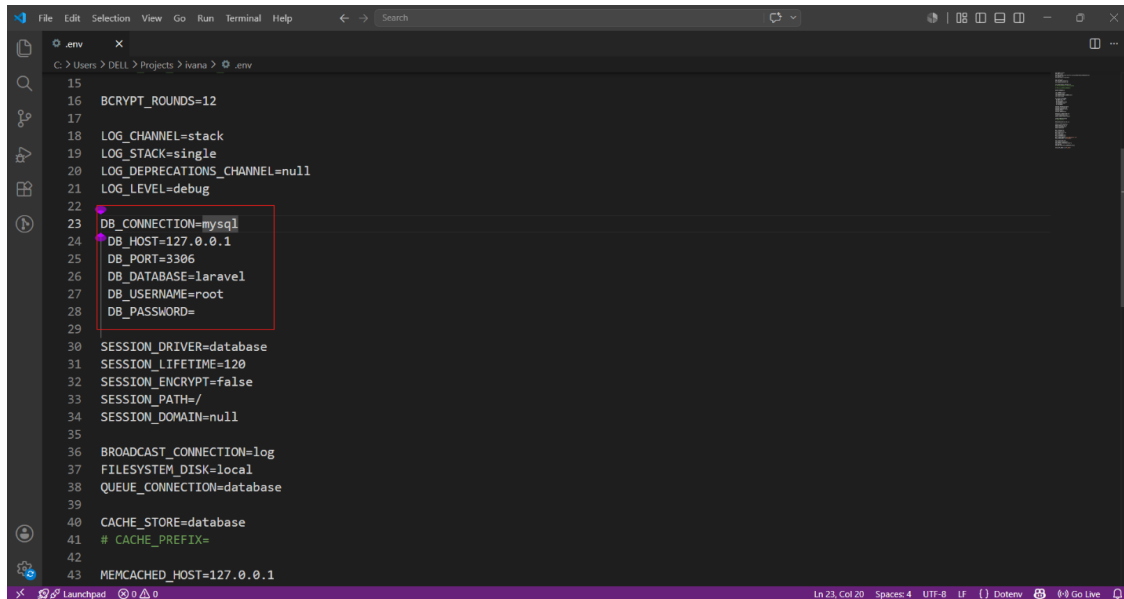


Fig 5.2: .env file showing MySQL connection configuration (highlighted section)

6. Building the Laravel MVC Structure

6.1 Creating the Model, Migration and Controller

Laravel's Artisan command-line tool was used to generate the Student Model, Migration file, and Resource Controller simultaneously using a single command. This is one of Laravel's most powerful features — it scaffolds the entire MVC structure in one step.

```
php artisan make:model Student -mcr
```

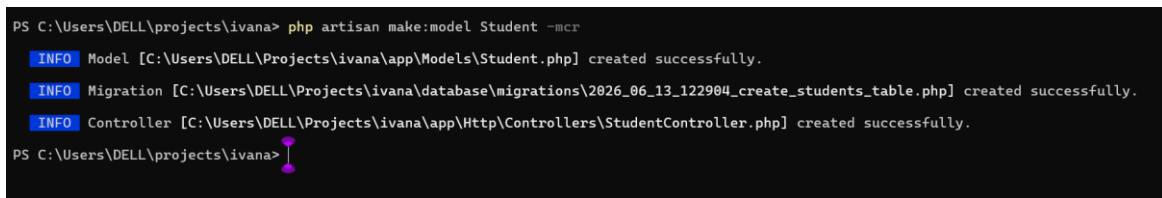


Fig 6.1: Output showing successful creation of Model, Migration and Controller

6.2 Defining the Database Schema (Migration)

The generated migration file was edited to define the structure of the students table. Two columns — name (string) and email (string, unique) — were added alongside the default id and timestamps columns.

```
Schema::create('students', function (Blueprint $table) {
    $table->id();
```

```

$table->string('name');
$table->string('email')->unique();
$table->timestamps();
});

```

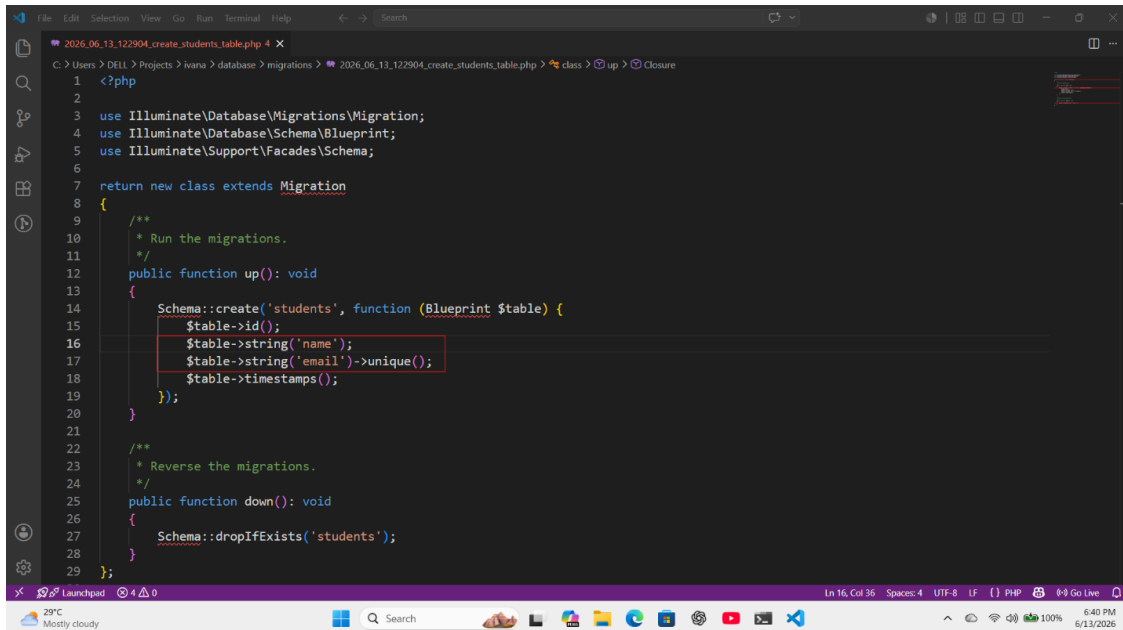


Fig 6.2: Migration file with name and email columns defined (highlighted)

6.3 Running Migrations on MySQL

The configuration cache was cleared first using `php artisan config:clear`, then `php artisan migrate` was executed to run all migration files and create the required tables in the MySQL database.

```

php artisan config:clear
php artisan migrate

```

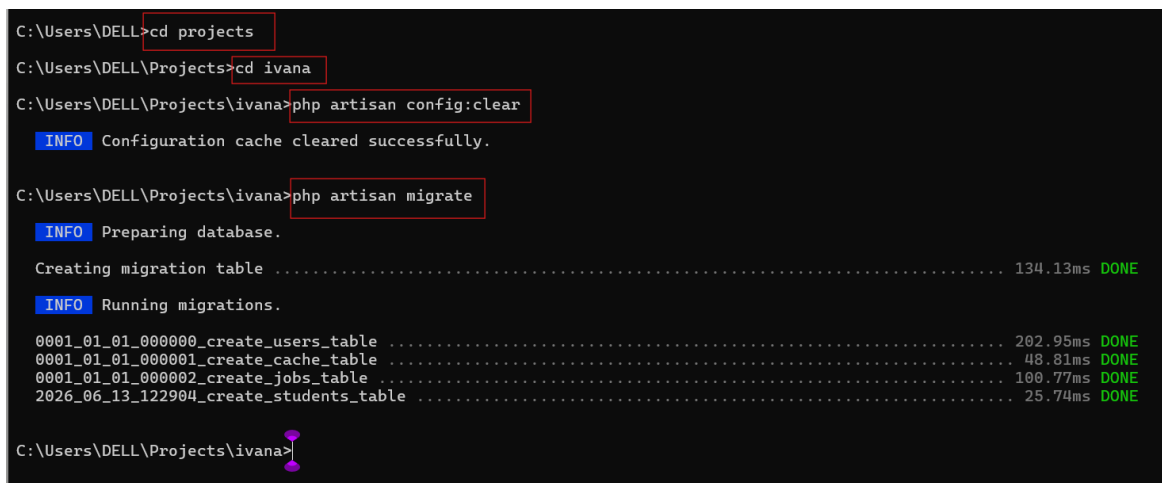


Fig 6.3: Successful migration output showing all tables created in MySQL

6.4 Defining Resource Routes

A single resource route was added to `routes/web.php`. Laravel's `Route::resource()` method automatically generates all seven standard RESTful routes (index, create, store, show, edit, update, destroy) needed for CRUD operations.

```
Route::resource('students', StudentController::class);
```

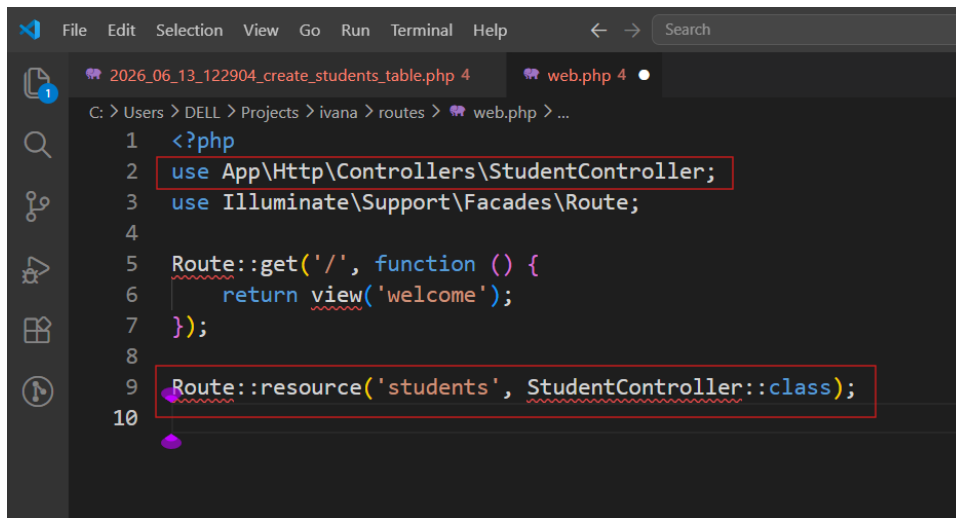


Fig 6.4: web.php showing the resource route for StudentController

6.5 Implementing the Controller

The StudentController.php was implemented with the CRUD logic. The index() method retrieves all student records using Student::all(), create() returns the create form view, store() validates and saves a new record using Student::create(), edit() and update() handle modification of existing records, and destroy() removes a record using \$student->delete().

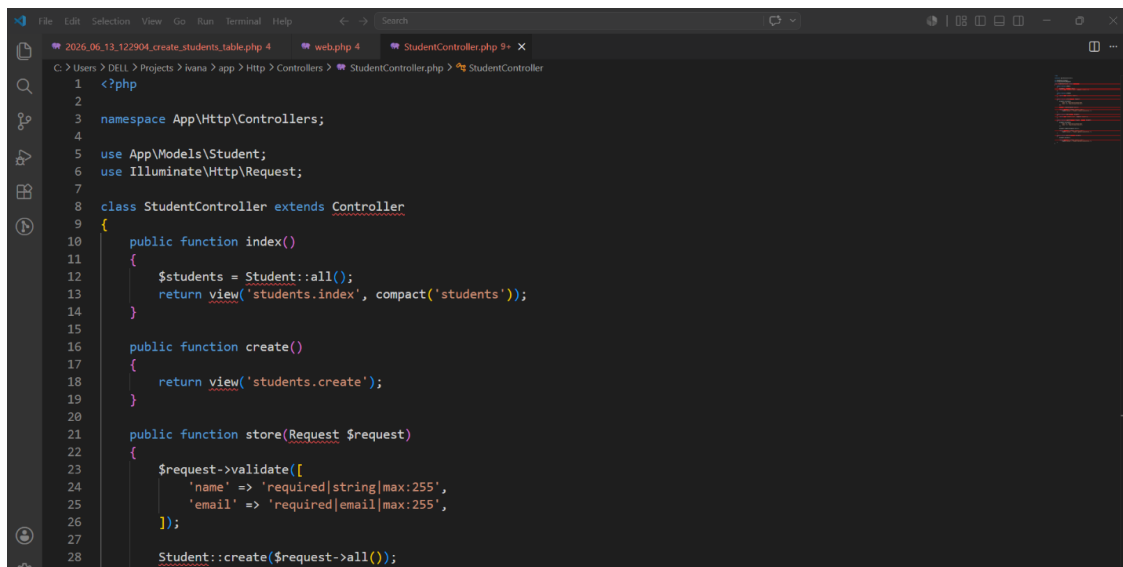


Fig 6.5: StudentController.php showing the CRUD method implementations

6.6 Creating Blade View Files

Three Blade view files were created inside resources/views/students/ to provide the HTML interface for each CRUD operation. The index.blade.php lists all students with Edit and Delete options, create.blade.php provides a form to add a new student, and edit.blade.php provides a pre-filled form for updating an existing record.

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Student List</title>
5 </head>
6 <body style="font-family: Arial; margin: 40px;">
7
8   <h1>Student Management (CRUD)</h1>
9
10  @if(session('success'))
11    <p style="color: green; font-weight: bold;">{{ session('success') }}
12  @endif
13
14  <a href="{{ route('students.create') }}">
15    <button style="padding: 8px 16px; margin-bottom: 15px;">+ Add New Student
16  </a>
17
18  <table border="1" cellpadding="10" cellspacing="0" style="width: 100%; text-align: center;">
19    <tr style="background-color: #f0f0f0;">
20      <th style="width: 10%;">ID</th>
21      <th style="width: 30%;">Name</th>
22      <th style="width: 30%;">Email</th>
23      <th style="width: 30%;">Actions</th>
24    </tr>
25    @foreach($students as $student)
26      <tr>
27        <td>{{ $student->id }}</td>
28        <td>{{ $student->name }}</td>
29        <td>{{ $student->email }}</td>

```

Fig 6.6(a): index.blade.php — Student list view with CRUD action buttons

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Add Student</title>
5 </head>
6 <body style="font-family: Arial; margin: 40px;">
7
8   <h1>Add New Student</h1>
9
10  @if($errors->any())
11    <div style="color: red;">
12      <ul>
13        @foreach($errors->all() as $error)
14          <li>{{ $error }}</li>
15        @endforeach
16      </ul>
17    </div>
18  @endif
19
20  <form action="{{ route('students.store') }}" method="POST">
21    @csrf
22
23    <label>Name:</label><br>
24    <input type="text" name="name" value="{{ old('name') }}"><br>
25
26    <label>Email:</label><br>
27    <input type="email" name="email" value="{{ old('email') }}"><br>
28
29    <button type="submit">Save</button>

```

Fig 6.6(b): create.blade.php — Form for adding a new student

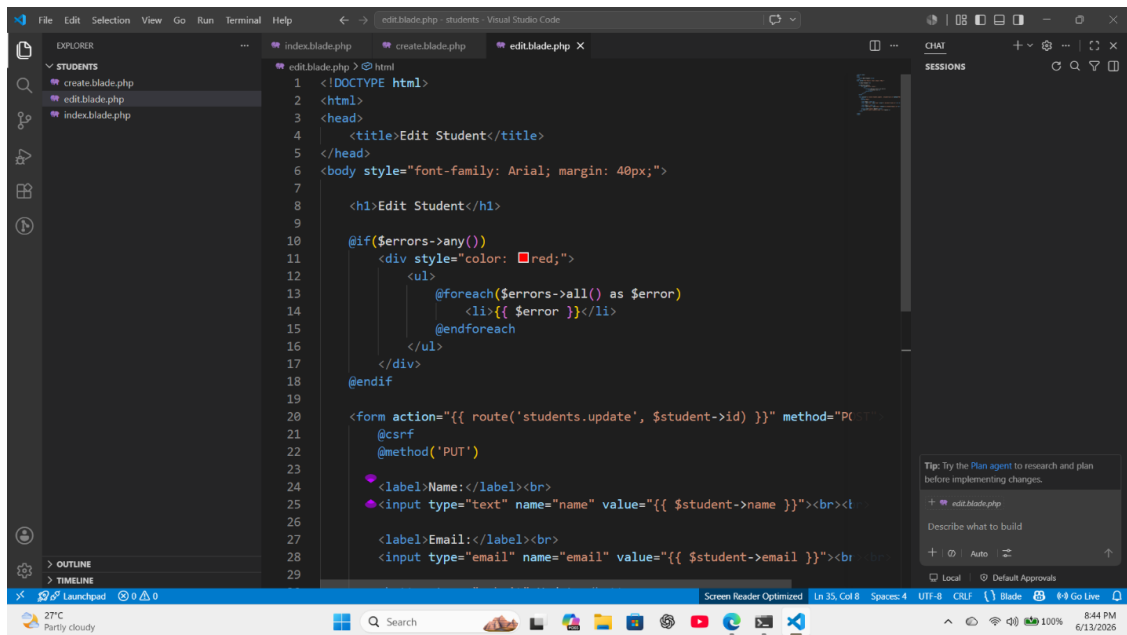


Fig 6.6(c): edit.blade.php — Form for editing an existing student

7. Verifying Tables in phpMyAdmin

After running the migrations, phpMyAdmin was refreshed to confirm that all tables had been successfully created inside the 'laravel' MySQL database. The students table along with default Laravel tables such as cache, jobs, users, sessions and migrations were all visible, confirming that the Laravel-MySQL connection was working correctly.

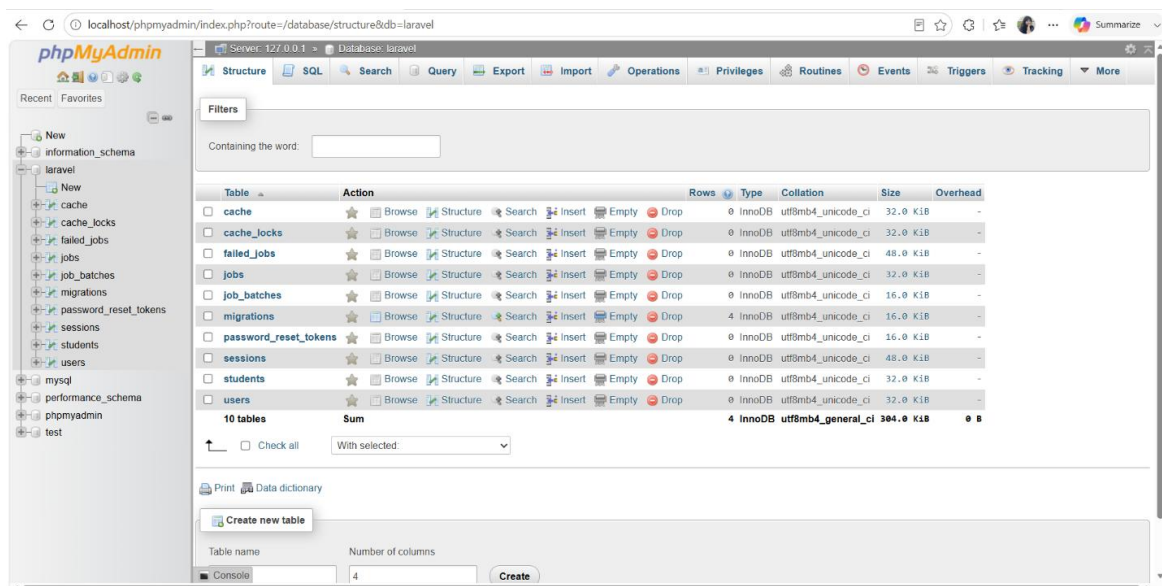


Fig 7.1: phpMyAdmin showing all migration tables created in the 'laravel' MySQL database

8. Testing CRUD Operations

The Laravel development server was started and the application was accessed through the browser. Each CRUD operation was tested and verified both in the browser interface and in

phpMyAdmin to confirm that data was being correctly stored, retrieved, modified, and removed from the MySQL database.

```
C:\Users\DELL\Projects\ivana>php artisan serve  
  
INFO Server running on [http://127.0.0.1:8000].  
  
Press Ctrl+C to stop the server
```

Fig 8.1: Laravel development server running on http://127.0.0.1:8000

8.1 Read Operation

Navigating to <http://127.0.0.1:8000/students> displayed the student list page with an empty table, since no records existed yet in the MySQL database. This confirms the `index()` controller method and `index.blade.php` view are functioning correctly, executing `Student::all()` which runs a `SELECT * FROM students` query on the MySQL database.

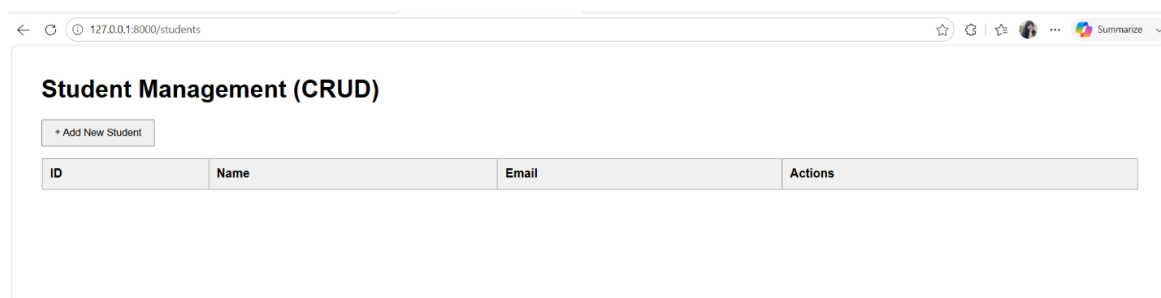


Fig 8.2: Student list page showing an empty table (Read operation before any data is added)

8.2 Create Operation

Clicking 'Add New Student' opened the create form. The name 'Ivana Islam' and email '2304034@uftb.ac.bd' were entered and submitted. The `store()` method validated the input and executed `Student::create()` which inserted the new record into the MySQL students table via an INSERT SQL query.

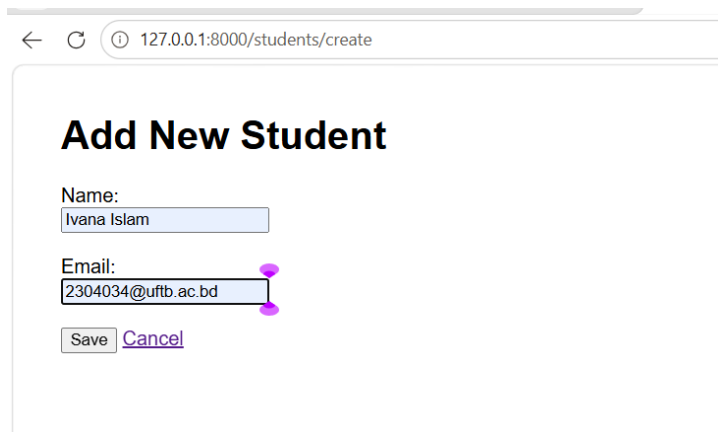


Fig 8.3(a): Add New Student form filled with sample data

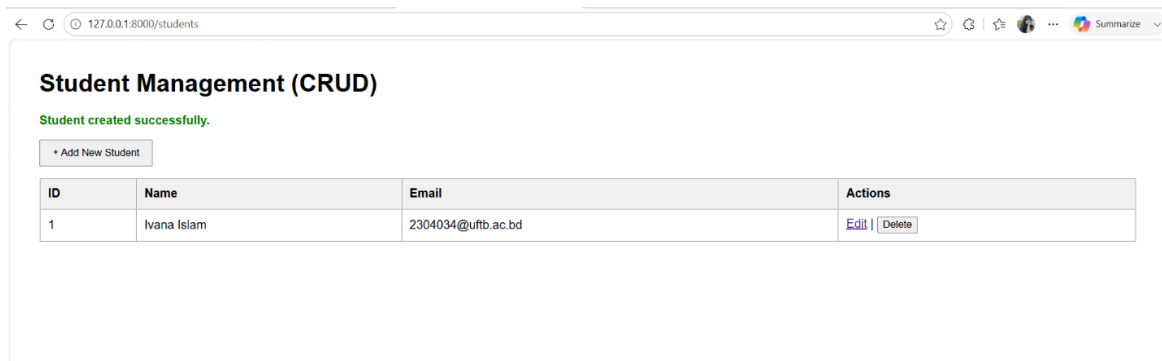


Fig 8.3(b): Student created successfully and displayed in the list

8.3 Verifying Create in phpMyAdmin

After the student was added through the browser, phpMyAdmin was opened to directly inspect the MySQL students table. The new record was visible with all fields correctly populated: id=1, name='Ivana Islam', email='2304034@uftb.ac.bd', along with the auto-generated created_at and updated_at timestamps. phpMyAdmin displayed this data by executing `SELECT * FROM students`, confirming the record was successfully stored in MySQL.

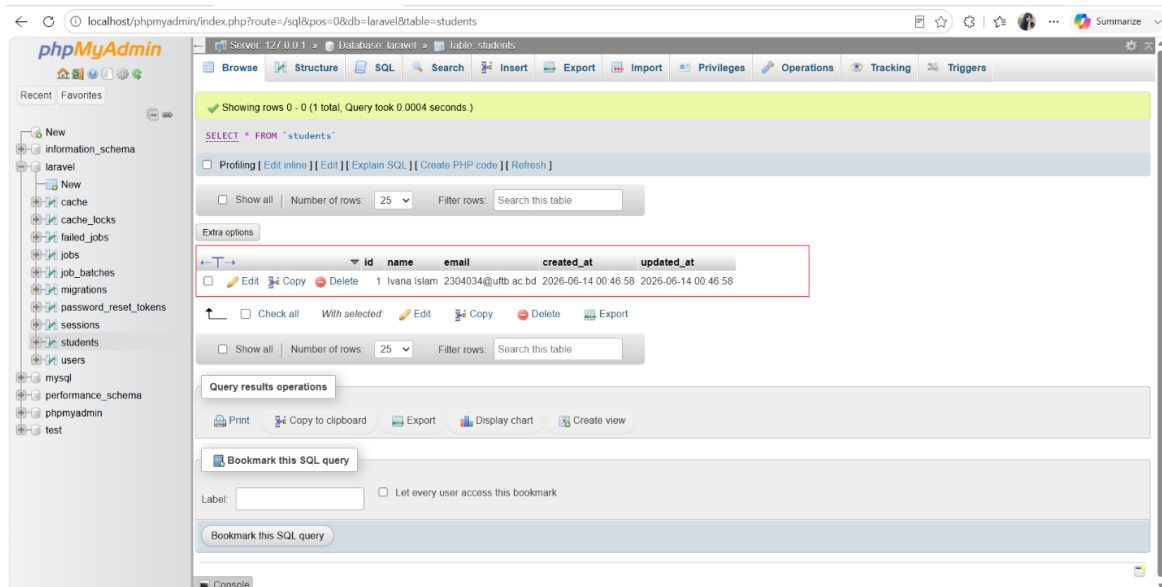


Fig 8.4: phpMyAdmin confirming the new student record stored in the MySQL 'students' table

8.4 Update Operation

The Edit link was clicked for the student record, opening the edit form pre-populated with the existing data. The name was changed to 'Borna' and the Update button was clicked. The `update()` controller method then executed `$student->update()` which ran an `UPDATE` SQL query on the MySQL database to modify the record.

← ↻ ⓘ 127.0.0.1:8000/students/1/edit

Edit Student

Name:

Email:

[Cancel](#)

Fig 8.5(a): Edit Student form pre-filled with existing data, name changed to 'Borna'

← ↻ ⓘ 127.0.0.1:8000/students

Student Management (CRUD)

Student updated successfully.

ID	Name	Email	Actions
1	Borna	2304034@uftb.ac.bd	Edit Delete

Fig 8.5(b): Student record successfully updated, new name visible in the list

8.5 Delete Operation

The Delete button was clicked for the student record, triggering a JavaScript confirmation dialog ('Are you sure?'). After confirming, a DELETE HTTP request was sent to the destroy() controller method, which executed \$student->delete() to permanently remove the record from the MySQL students table via a DELETE SQL query.

← ↻ ⓘ 127.0.0.1:8000/students

Student Management (CRUD)

Student updated successfully.

127.0.0.1:8000 says
Are you sure?

ID	Name	Email	Actions
1	Borna	2304034@uftb.ac.bd	Edit Delete

Fig 8.6(a): JavaScript confirmation dialog before deleting the student record

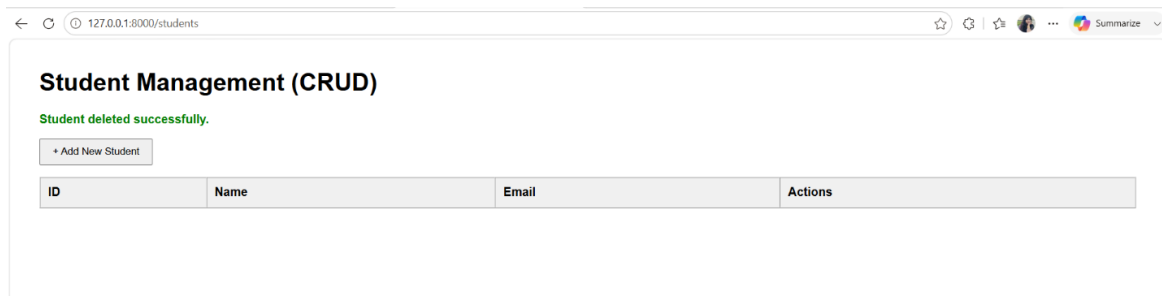


Fig 8.6(b): Record successfully deleted, student list returns to empty state

9. Conclusion

This lab successfully demonstrated the implementation of all four CRUD operations — Create, Read, Update, and Delete — in a Laravel web application connected to a MySQL database running on XAMPP. The complete development workflow was followed: setting up the MySQL environment via XAMPP and phpMyAdmin, configuring the Laravel .env file, building the MVC structure using Artisan commands, implementing the controller logic, creating Blade views, and testing all operations through the browser. Every operation was additionally verified in phpMyAdmin, confirming that data was correctly managed in the MySQL students table. This lab provided practical experience with a full-stack Laravel-MySQL development environment that closely reflects real-world web application development workflows.